Individual Assignment 1
Basic Matrix Multiplication in Different Languages

Adrián Rodríguez Pérez

Big Data · Grado en Ciencia e Ingeniería de Datos · Universidad de Las Palmas de Gran Canaria · 2026-2027

Abstract. The same i-j-k triple-loop multiplication of two n × n `float64` matrices was implemented in C, Java and Python and run on one laptop (AMD Ryzen 9 7945HX, Windows 11) for n = 100 to 2,800, with shared input files, a kernel-only timer, warm-up runs, five measured repetitions per configuration, and a 60 s per-run budget. All three implementations produce bit-identical results that agree with NumPy's matrix product within a rounding-error tolerance. Runtime grows close to the cubic model (fitted exponents 3.08 for C, 3.21 for Java, 3.15 for Python for n ≥ 300). Under these settings, Java took 1.1–1.7× the C time up to n = 1,000 and 2.3× at n = 1,400, and Python took 72–85× the C time. The largest sizes completed within budget were n = 2,000 (C), 1,400 (Java) and 700 (Python). A supplementary diagnostic shows that most of the Java–C gap, including a sharp slowdown above n ≈ 1,700, comes from the `double[][]` storage layout rather than from the JIT compiler: a Java version with a flat array runs at 0.95–1.18× the C time. Peak working set grows as n^2 with about 8 B per element in C, 8–15 B in Java (plus a ≈ 47 MB runtime baseline) and about 39 B in Python. These conclusions apply only to the tested implementations, settings and machine.

1. Introduction and objective

Matrix multiplication is a core kernel of data processing and machine learning, and its cost grows quickly with size. This study compares the textbook algorithm written the same way in three languages commonly used in data engineering.

Research question. How do the runtime and peak memory of the basic triple-loop matrix multiplication grow with the matrix size n in C, Java and Python?

Factor changed: n, from 100 to 2,800, and the implementation language. **Outcomes:** kernel time per multiplication and peak working set of the process.

Hypotheses (stated before measuring). H1: every implementation performs n^3 multiply-adds, so time should grow approximately as n^3 (×8 per doubling of n), with constant factors set by the language runtime. H2: compiled C and JIT-compiled Java should be within a small factor of each other, while the CPython interpreter should be one to two orders of magnitude slower. H3: memory should grow as n^2, with C close to the $3 \times 8n^2$ bytes of raw `float64` storage, Java adding a fixed JVM baseline, and Python several times more per element because each value is a separate heap object.

2. Background

For $C = AB$ with n × n matrices (matrices are written in italics; plain C is the programming language), the classical algorithm computes each of the n^2 entries as a dot product of length n, giving n^3 multiplications and n^3 additions, i.e. $\Theta(n^3)$ time and $\Theta(n^2)$ storage. In the i-j-k order used here, the innermost loop reads row i of A sequentially but column j of B with a stride of n elements, so consecutive iterations touch different cache lines and, for large n, different virtual-memory pages. Locality, the cache hierarchy and the translation lookaside buffer (TLB) therefore affect the constant factor of the cubic model.

The three languages execute the same loop very differently. GCC compiles C ahead of time to machine code. Java is compiled to bytecode and then, during execution, by HotSpot's tiered JIT compilers. Early runs are partly interpreted, which is why warm-up runs are needed, and every array access is bounds-checked as the

language specification requires. CPython interprets bytecode: version 3.12 specialises instructions adaptively, but each arithmetic operation still works on heap-allocated `float` objects.

3. Methodology

3.1 Implementations

All three kernels use the same loop order (i, then j, then k), the same scalar accumulator (`sum = 0.0; sum += a[i][k]·b[k][j]; c[i][j] = sum`), IEEE-754 binary64 (`double`/`float`) arithmetic and a single thread. No library multiplication routine is called. Row i of *A* and of the result matrix *C* is read through a local reference in all three, which does not change the work.

Language	Storage layout of each matrix	Build / run settings (fixed for all sizes)
C (`src/c/matmul.c`)	One contiguous row-major block of n^2 `double` (8 B each), indexed `k*n + j`	GCC 16.2.0 (MSYS2 UCRT64), `-O2 -std=c11 -ffp-contract=off`; no `-march`, so no FMA. GCC's vectorisation report (`results/c_vectorization.txt`) confirms the kernel loops are *not* vectorised (reason given: strided load of B).
Java (`src/java/MatMul.java`)	`double[n][n]`: an array of n references to separate row arrays, each with a 16 B object header and an implicit length used for bounds checks	OpenJDK 21.0.10 (Temurin), `javac --release 21`, `java -Xmx4g`; default G1 garbage collector and tiered JIT. 3 warm-up runs in the same JVM before measuring.
Python (`src/python/matmul.py`)	List of n lists, each holding n pointers (8 B) to separate `float` objects (24 B each)	CPython 3.12.0 (64-bit), pure Python, no NumPy in the kernel. 1 warm-up run.

Unavoidable differences are the storage layouts above, Java's bounds checks, and the JVM's background JIT-compiler and GC threads. The multiplication itself is single-threaded in all three. Because the arithmetic and evaluation order are identical, the three outputs are expected to be bit-identical, and they are (see Section 4.1).

3.2 Inputs

For each n, A and B are generated once by `scripts/gen_inputs.py` with NumPy's PCG64 generator seeded with `[2026, n]`, uniform in [−1, 1), and saved as raw little-endian row-major `float64` files. All three programs read the same files, so every language multiplies exactly the same logical matrices. `results/inputs_manifest.csv` records the SHA-256 of every file. The size sequence 100, 200, 300, 400, 500, 700, 1,000, 1,400, 2,000, 2,800 grows by about √2 at large n. Powers of two (e.g. 512, 1024, 2048) were avoided: with such sizes, consecutive elements of a column of B are a power-of-two number of bytes apart and compete for the same few cache sets, which can produce slowdowns at those specific sizes that are unrelated to the language comparison.

3.3 Environment

Item	Value
CPU	AMD Ryzen 9 7945HX (Zen 4), 16 physical cores, 32 logical processors; 1 MB L2 per core, 2 × 32 MB L3
RAM / OS	32 GB (31.2 GiB) / Windows 11 Enterprise LTSC, build 10.0.26200 (laptop; power plan and CPU frequency not controlled or recorded)
Threads	1 kernel thread per implementation; no affinity set; the JVM also runs JIT and G1 service threads; NumPy (inputs and validation only) never runs during timing

Tools	Python 3.12.0, NumPy 2.3.5, pandas 3.0.1, matplotlib 3.10.8, psutil 7.2.2; full listing in `results/environment.json`

3.4 Measurement protocol

- **Timing boundary: kernel only.** Loading the inputs and allocating and zero-initialising the result matrix *C* happen before the timer starts. The timed region is exactly one call of the triple loop. Monotonic timers: `QueryPerformanceCounter` (C), `System.nanoTime()` (Java), `time.perf_counter_ns()` (Python).
- **Process and repetitions.** Each (language, n) configuration runs in a fresh process that performs warm-up runs (C 1, Java 3, Python 1) followed by 5 measured runs. Warm-up runs are recorded in the raw data but excluded from the statistics. The summary is the median, and variability is the interquartile range (IQR).
- **Result consumption.** After every run the program prints a *checksum*, the sum of all entries of *C*, which is stored in the raw data. Printing it makes the result observable, so the compiler cannot eliminate the computation as unused. Correctness is not judged from the checksum but from the full output matrix: the final *C* is written to disk and checked as described in Section 3.5.
- **Memory metric: peak working set of the whole process**, i.e. the maximum amount of physical RAM the process occupied at any moment of its execution [1]. It counts only memory actually loaded in RAM, not memory that was merely reserved (Windows `PeakWorkingSetSize`, read with psutil's `peak_wset` by the driver). Each program waits on stdin after its last run, so the value is read while the process is still alive. The same metric and tool are used for all three languages. It includes the runtime itself (interpreter, JVM, libraries), so the lowest size shows each runtime's baseline. As a second reference, the raw storage estimate $3 \times 8n^2$ B is reported.
- **Budgets, declared beforehand.** 60 s per run (a 30 s allowance is added before the first run for input loading) and 4 GB of estimated matrix memory. A run that exceeds the time budget is killed and recorded as a timeout without a time value, and larger sizes for that language are not attempted.

3.5 Correctness checks

1. Each implementation's self-test (`--selftest`) checks, with exact equality, a hand-computed 2 × 2 product, a 3 × 3 product, A·I = A, I·A = A and A·0 = 0. Small integers are exact in binary64.
2. Every benchmark output is compared with NumPy's matrix product (`A @ B`). NumPy adds the n products of each entry in a different order, and floating-point rounding makes the results differ slightly, so an exact comparison is not possible. Each entry is accepted if

   $$|C_{ij} - C_{\mathrm{ref},ij}| \le 2\,\gamma_n\,(|A|\,|B|)_{ij}, \quad \gamma_n = nu / (1 - nu), \quad u = 2^{-53}$$

   where $|A|$ and $|B|$ contain the absolute values of *A* and *B*, and u is the largest relative rounding error of a single float64 operation. $\gamma_n \approx n{\cdot}u$ is the worst-case relative error accumulated over the n additions of one entry (about 1.1×10^{-13} for n = 1,000), so the tolerance grows with n and with the size of the entry, as rounding error does. The factor 2 covers the rounding error of the NumPy result as well.
3. The outputs of Java and Python are compared bit by bit with the output of the C implementation, and the checksum of every run is required to be identical.

4. Results

4.1 Correctness

All 65 checks in `results/validation.csv` pass. All self-tests pass in the three languages. The largest error with respect to NumPy is 2.98×10^{-13} (C, n = 2,000), and the largest ratio of error to tolerance is 9.7×10^{-3} (n = 100), i.e. no entry used more than 1 % of its tolerance. Java and Python outputs are bit-identical to C at every size they completed, and every run of a configuration produced the same checksum.

4.2 Runtime

n	C median [IQR] (s)	Java median [IQR] (s)	Python median [IQR] (s)	Java / C	Python / C
100	0.000887 [0.000057]	0.00111 [0.00023]	0.0752 [0.0004]	1.2×	85×
200	0.00831 [0.00007]	0.00939 [0.00006]	0.596 [0.005]	1.1×	72×
300	0.0298 [0.0002]	0.0473 [0.0012]	2.15 [0.02]	1.6×	72×
400	0.0739 [0.0007]	0.124 [0.001]	5.34 [0.04]	1.7×	72×
500	0.145 [0.001]	0.246 [0.002]	10.9 [0.35]	1.7×	75×
700	0.407 [0.001]	0.681 [0.006]	30.8 [0.22]	1.7×	76×
1,000	1.19 [0.01]	1.90 [0.01]	timeout	1.6×	—
1,400	3.32 [0.02]	7.62 [0.07]	not attempted	2.3×	—
2,000	10.90 [0.01]	timeout	not attempted	—	—
2,800	timeout	not attempted	not attempted	—	—

Table 1. Kernel time per multiplication: median of 5 measured runs, IQR in brackets (e.g. 0.000887 [0.000057] = median 0.000887 s, IQR 0.000057 s); the ratio columns divide medians. Measured after the warm-up runs of Section 3.4. Raw values: `results/raw_times.csv`; full summary with quartiles, minimum and maximum: `results/summary.csv`.

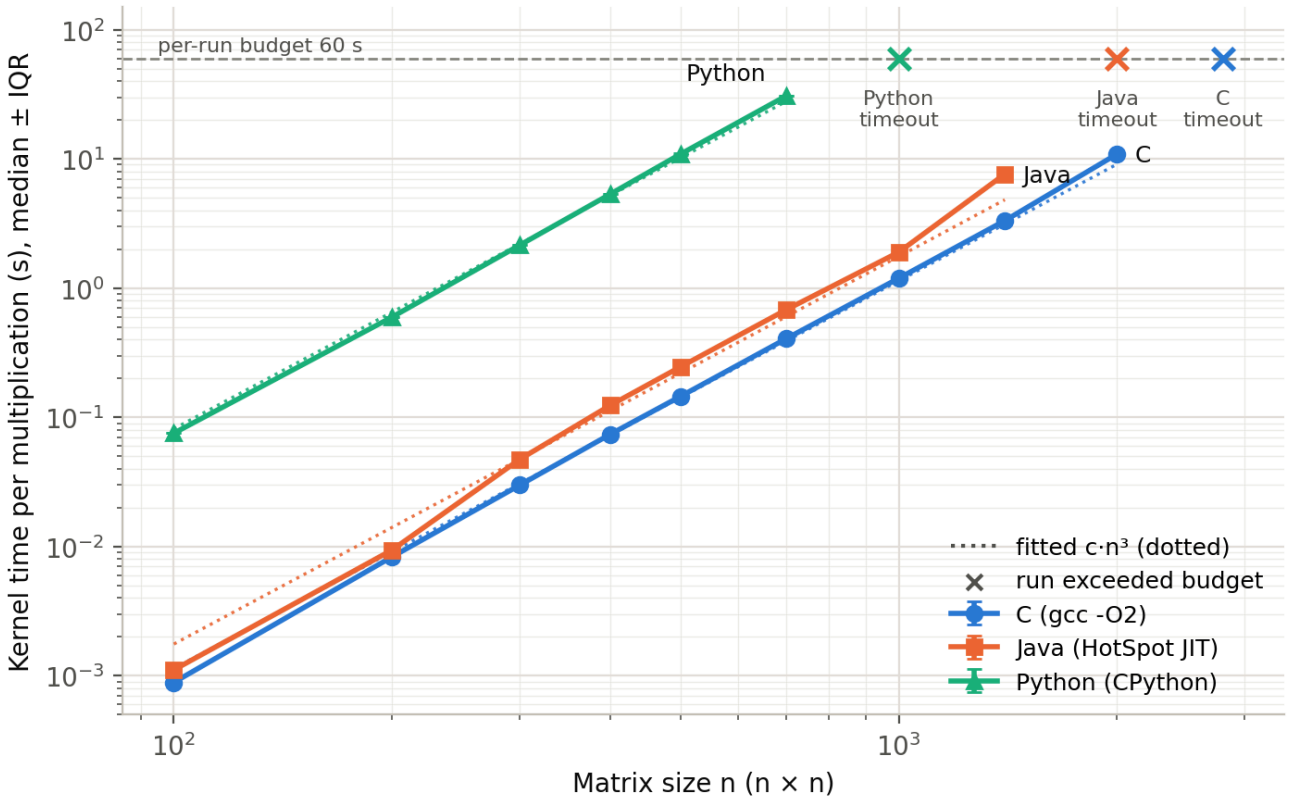


Figure 1. Kernel time against n on log-log axes (median, error bars = IQR, mostly smaller than the markers). Dotted lines: $c \cdot n^3$ fitted to each language with the exponent fixed at 3. Crosses: the first size whose run exceeded the 60 s budget.

The variability is small. The relative IQR (IQR/median) is at most 1.02 % in 19 of the 23 completed configurations and 2.5–6.4 % in three (C n = 100, Java n = 300, Python n = 500). It reaches 20 % only for Java at n = 100, a 1 ms task where JIT recompilation is still visible. A least-squares fit of log t against log n for n ≥ 300 gives exponents of 3.08 (C), 3.21 (Java) and 3.15 (Python), all close to the cubic model and all slightly above 3. Figure 2 shows why: the time per multiply-add, t/n^3, would be flat if the model held exactly, but it rises with n in every language.

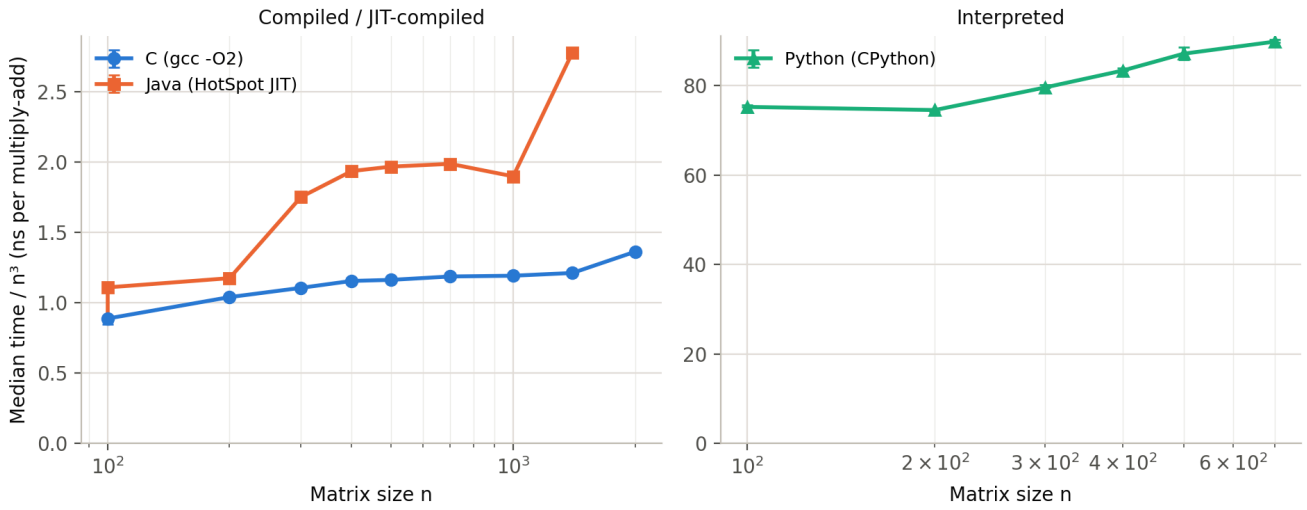


Figure 2. Median time divided by n^3, i.e. nanoseconds per multiply-add. Left: C and Java. Right: Python (note the different scale). A flat line would mean exact n^3 growth.

C rises from 0.89 to 1.36 ns per multiply-add between n = 100 and 2,000. Java rises from 1.11 ns to about 1.9–2.0 ns between n = 300 and 1,000, then jumps to 2.78 ns at n = 1,400. Python rises from 75 to 90 ns.

4.3 Limits within the budget

Language	Largest size completed within budget	Largest size attempted and outcome
C	2,000 (10.9 s)	2,800: the warm-up run took 61.4 s (> 60 s); the process was killed during the next run
Java	1,400 (7.6 s)	2,000: the first warm-up run took 65.0 s (> 60 s); the process was killed during the next run
Python	700 (30.8 s)	1,000: no run finished within the 90 s allowed for the first run (60 s + 30 s loading allowance)

The over-budget times above are single unmeasured observations, kept in the raw data but not used as medians. The memory budget was never the limiting factor: at n = 2,800 the estimated storage is 188 MB (C, Java) and 753 MB (Python), far below 4 GB.

4.4 Memory

n	$3 \times 8n^2$ estimate	C peak working set	Java peak working set	Python peak working set
100	0.24	4.7	47.4	13.4
300	2.2	6.6	49.8	23.4
500	6.0	10.4	53.8	42.9
700	11.8	16.2	59.7	71.7
1,000	24.0	28.4	92.5	—
1,400	47.0	51.5	114.7	—
2,000	96.0	100.4	—	—

Table 2. Peak working set of the whole process (MB, 10^6 B; one value per configuration, measured by psutil `peak_wset`), against the raw storage of three float64 matrices. Source: `results/runs.csv`.

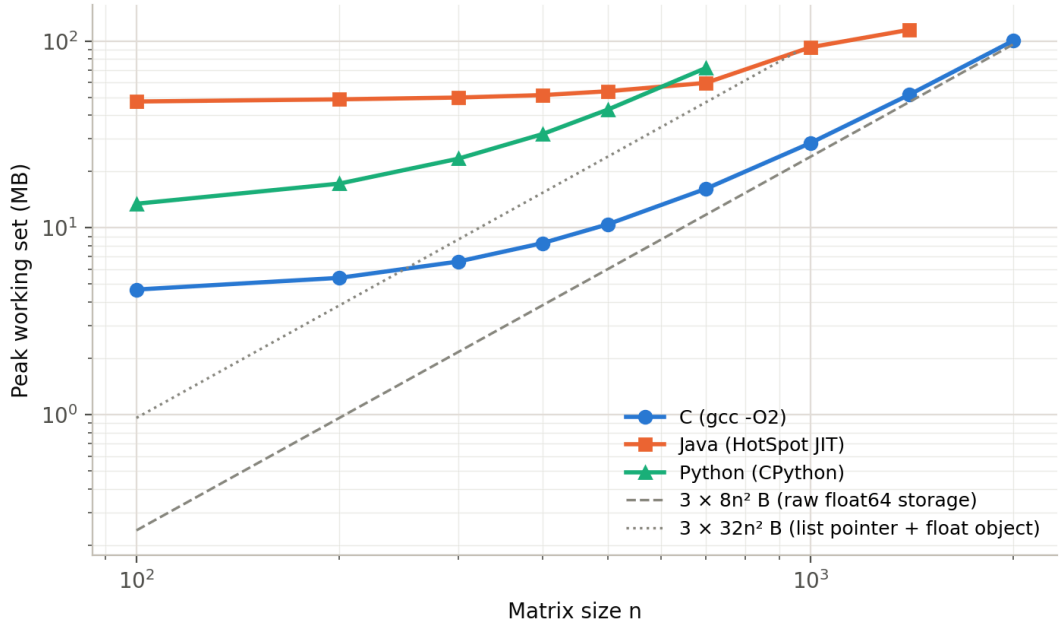


Figure 3. Peak working set against n (log-log). Dashed: 3 × 8n² B of raw float64 storage. Dotted: 3 × 32n² B, a pointer plus a float object per element.

Subtracting each runtime's baseline (its value at n = 100) and dividing by the $3n^2$ stored elements gives the marginal cost per element: 7.9–8.0 B in C (n = 1,000 and 2,000), 8.3 B (n = 700), 15.0 B (n = 1,000) and 11.4 B (n = 1,400) in Java, and 39.2–39.6 B in Python (n = 500 and 700). At the largest size common to all three, n = 700, the peak working sets are 16 MB (C), 60 MB (Java) and 72 MB (Python).

4.5 Java warm-up

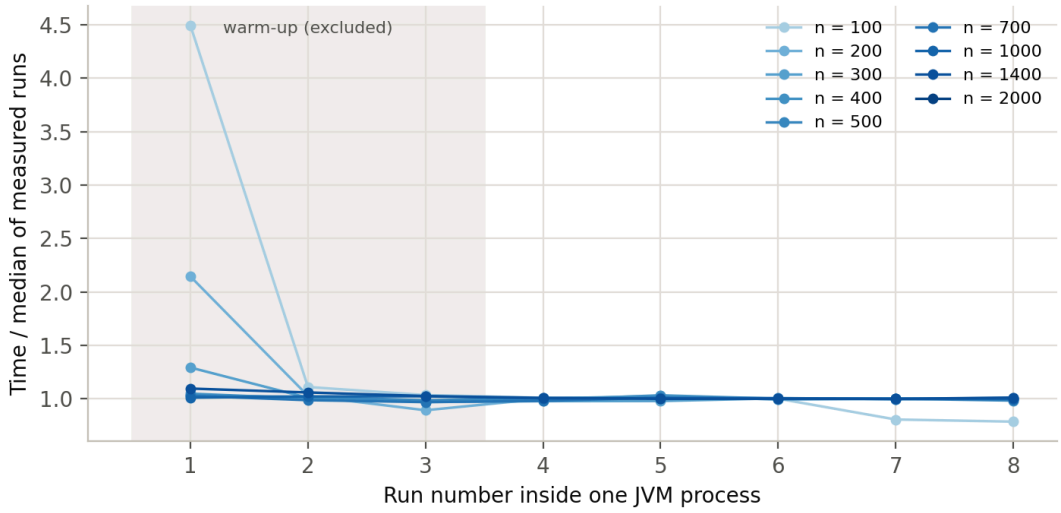


Figure 4. Each Java run's time divided by the median of the measured runs (runs 4–8), by run number inside one JVM. The shaded area marks the excluded warm-up runs.

The first Java run is 4.5× slower than the measured median at n = 100 and 2.1× at n = 200, but within 10 % for n ≥ 400, where one run is long enough for the JIT to compile the loop while it executes. Three warm-up runs are sufficient: after run 3 every size is stable. In C the first run is within 5 % of the median at every size (column `first_warmup_s` in `results/summary.csv`). As in C, Python does not depend on its warm-up run (one was still performed and excluded): it has no runtime compilation, so its first run is not systematically slower than the later ones.

4.6 Supplementary diagnostic: storage layout (separate experiment)

The growing Java/C ratio in Table 1 had no explanation from the main data, so an additional experiment was run. It is *not* part of the controlled comparison. `src/probe/` contains the same i-j-k kernel restricted to the first 16 rows of *C*, which keeps the access pattern to B identical while making large n cheap. It measures three layouts of B: C contiguous, Java `double[n][n]` (as in the main study), and Java with one flat `double[n·n]` indexed `k*n + j`, exactly like C. Inputs come from the same xorshift generator in both languages; there were 3 warm-up runs and 5 measured runs per point. The script is `scripts/run_probe.py` and the data are in `results/probe_times.csv`.

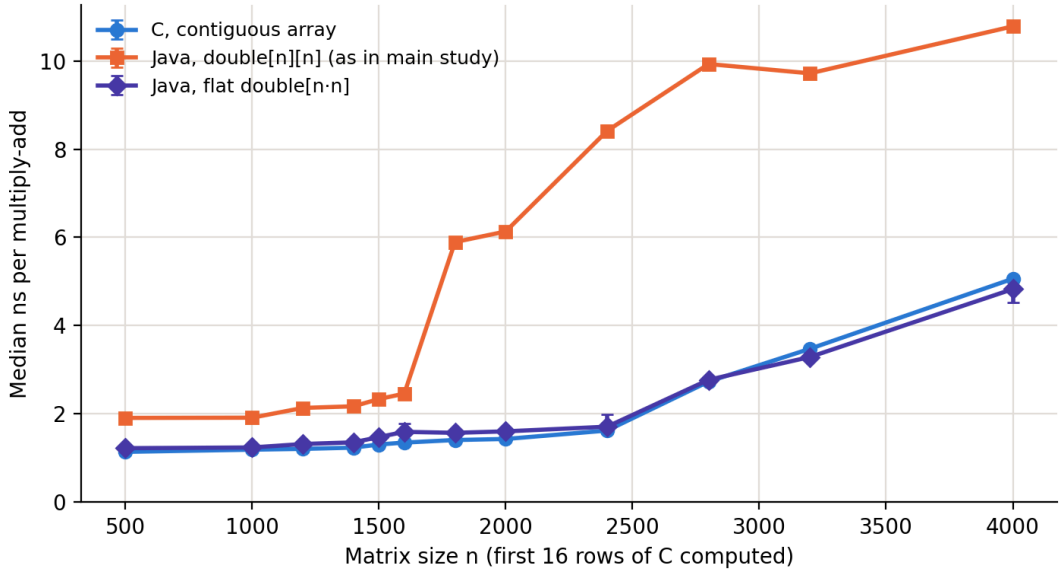


Figure 5. Median time per multiply-add for the 16-row kernel (error bars = IQR) for three storage layouts of B.

Java with a flat array runs at 0.95–1.18× the C time at every size from 500 to 4,000 (for example 1.24 vs 1.19 ns at n = 1,000, and 1.61 vs 1.43 ns at n = 2,000). Java with `double[][]` costs 1.9 ns already at n = 500 and jumps from 2.5 ns (n = 1,600) to 5.9 ns (n = 1,800), reaching 9.9 ns at n = 2,800. Every layout slows down beyond n ≈ 2,400 (C: 1.63 → 2.73 ns from n = 2,400 to 2,800), which is consistent with the C timeout at n = 2,800 in the main study.

5. Discussion

Answer to the question. On this machine, runtime grows close to n^3 in all three implementations (H1 holds, with exponents 3.08–3.21). The language changes the constant factor c in $t \approx c \cdot n^3$ (the time per multiply-add) far more than the exponent. Python's constant is about 75× that of C at every common size, and Java's is 1.1–2.3× and grows with n. Memory grows as n^2, with per-element costs of about 8 B (C), 8–15 B (Java) and 39 B (Python) on top of very different runtime baselines (H3 holds). H2 holds for Python but only partly for Java: Java is near C at small n, but the gap widens with size.

Why the exponents exceed 3. The model counts arithmetic, not memory traffic. In the i-j-k order each step of the inner loop reads B in a new cache line (stride 8n bytes), so as n grows, the column of B and the B matrix (8 MB at n = 1,000, 32 MB at n = 2,000, 63 MB at n = 2,800) fall out of successive cache levels, and each multiply-add waits longer for data. This produces the gradual rise of C from 0.9 to 1.4 ns per multiply-add, and the steeper rise beyond n ≈ 2,400 in Figure 5, when B is well beyond one 32 MB L3 slice. Because the sum is carried in one variable, each addition also depends on the previous one, so the CPU cannot overlap them. The compiler could only speed up this chain by regrouping the additions, which would change the rounding; without `-ffast-math`, GCC keeps the exact order written in the code. In addition, the elements of a column of B

are not contiguous in memory. GCC's vectorisation report confirms that the loop is not vectorised and gives the strided access to B as the reason.

Interpreter effect (Python). The inner loop is 15 bytecode instructions per multiply-add, including three subscript operations, a multiplication and an addition that each allocate a new `float` object. At about 75 ns per multiply-add, interpretation and object handling dominate. The C/Python ratio stays at 72–85× because both pay the same cubic count, and only the cost per step differs. Python's cost per step also rises with n (75 → 90 ns), plausibly because the float objects of B are scattered in memory, so column access touches more cache lines than in C. This was not isolated experimentally.

Runtime and layout effects (Java). The warm-up data (Section 4.5) show that JIT compilation matters only for runs shorter than about 0.1 s and is removed by the warm-up protocol. The diagnostic of Section 4.6 shows that, once compiled, the JIT-compiled loop over a flat array runs almost as fast as GCC's code. The remaining gap therefore comes from `double[][]`. Each `b[k][j]` must first load the reference `b[k]`, then read the length of row k for the bounds check required by the language, and only then load the element. Each step therefore depends on two loads and, when j is large, touches two different memory pages: the row header and the element. A plausible explanation for the sudden jump between n = 1,600 and 1,800 is the TLB. A pass down one column touches about n pages in C but about 2n in `double[][]`, so the `double[][]` layout would exceed the number of page translations the TLB can hold at a smaller n than the contiguous layout, which matches a sudden jump in Java only. However, hardware performance counters were not measured, so this explanation is consistent with the evidence but not proven.

Memory interpretation. C matches its storage almost exactly (8.0 B per element plus a 4.7 MB process baseline). The JVM starts at 47 MB before any matrix exists, and its marginal cost varies between 8 and 15 B per element, plausibly because G1 sizes and touches heap regions according to its own policy, not only according to live data. Python's 39 B per element is above the 32 B of a pointer plus a float object. The difference is plausibly allocator overhead (pymalloc pools and arenas), but this was not verified. Peak working set counts resident pages, so it is a property of the whole process and should not be compared with an estimate of array storage as if they were the same quantity.

Limitations. (i) One machine, one OS, one run of the whole suite. Laptop CPU frequency (boost, temperature) was not fixed or recorded, although the small IQRs indicate stable conditions within each configuration. (ii) Configurations ran sequentially, so slow drifts between sizes cannot be excluded. (iii) The 60 s budget is an arbitrary design choice: the "largest feasible size" depends on it, and C at n = 2,800 missed it by only 1.4 s. (iv) Peak working set includes runtime baselines and depends on OS paging decisions, and the JVM's value depends on GC settings, which were left at their defaults (except `-Xmx4g`). (v) Only one compiler flag set, one JVM and one Python version were tested, so other settings (e.g. `-O3 -march=native`, PyPy, a different GC) could change the constants. (vi) The cache/TLB explanations are inferred from timing patterns, not from hardware counters.

6. Conclusions

Within the tested scope (Ryzen 9 7945HX laptop, Windows 11, GCC 16.2 `-O2`, OpenJDK 21 with default settings, CPython 3.12, float64, n = 100–2,800), the study establishes that:

- The three implementations compute exactly the same result, and the cubic model describes the growth of their runtime well (exponents 3.08–3.21). The excess over 3 is consistent with memory-hierarchy costs that grow with n.
- The constant factors differ greatly: Python needs about 75× the C time, which limits it to n = 700 within 60 s, against n = 2,000 for C.
- The Java–C gap (1.1–2.3×) is mainly a consequence of the `double[][]` layout and grows with n. A flat Java array reduces it to 0.95–1.18× in the diagnostic experiment. Most of the measured gap therefore reflects

how the matrix is stored rather than how fast the compiled code runs.

- Memory grows as n^2. The per-element cost is about 8 B in C, 8–15 B in Java and 39 B in Python, plus fixed baselines of about 5, 47 and 13 MB.

Unresolved: the exact hardware mechanism behind the Java slowdown above n ≈ 1,700 and the general slowdown above n ≈ 2,400 (TLB and cache counters would settle it), and how sensitive these results are to compiler, JVM and frequency settings. Assignment 2 will reuse the validated C or Java implementation and this dataset as the baseline for loop reordering and cache blocking, which directly target the column-wise access identified here.

7. References

1. Microsoft, "Working Set," Win32 memory management documentation. [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/memory/working-set

8. Reproducibility materials

Repository: https://github.com/AdriRP/BigData-Assignment01-Matrix-Multiplication. It contains this report, a README with exact build and run commands and versions, the source code (`src/`), the configuration (`config.json`), the scripts (`scripts/`), all raw and summarised measurements (`results/`) and the figures (`figures/`); the submitted `Assignment01_Adrian_Rodriguez_Materials.zip` has the same contents. The input matrices (≈ 300 MB) are not included: they are regenerated by `scripts/gen_inputs.py`, which checks them against `results/inputs_manifest.csv`. From the folder that contains `config.json`, the whole study is reproduced with:

```
pip install -r requirements.txt
python scripts/run_all.py        # full study (~25 min)
python scripts/run_probe.py      # layout diagnostic (~3 min)
```

This is the first assignment of the series, so no earlier implementation or dataset is reused. The `multiply` kernels and the input files defined here are the baseline for Assignment 2.